

Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

COMPTE RENDU

TP2: Requêtes, dépendances fonctionnelles et normalisation

Réalisé par:

ELGARANI Hasna

2024/2025

Exercice n°1 :

1. Afficher le nom du département qui a le budget le plus élevé.

```
SELECT dept_name
FROM department
WHERE budget IN (
    SELECT MAX(budget)
    FROM department
);
```

Results	Explain	Describe	Saved SQL	History
DEPT_NAME				
Finance				
1 rows returned in 0.01 seconds Download				

2. Afficher les salaires et les noms des enseignants qui gagnent plus que le salaire moyen.

```
SELECT name, salary
FROM teacher
WHERE salary > (SELECT AVG(salary) FROM teacher);
```

Results	Explain	Describe	Saved SQL	History
NAME				
SALARY				
Wu				
90000				
Einstein				
95000				
Gold				
87000				
Katz				
75000				
Singh				
80000				
Brandt				
92000				
Kim				
80000				
7 rows returned in 0.01 seconds Download				

3. Pour chaque enseignant, afficher tous les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant, ainsi que le nombre total de cours suivis par chaque étudiant, en utilisant la clause HAVING.

```
SELECT teacher.teacher_name, student.student_name, COUNT(*)
FROM teacher, student, takes, teaches
WHERE teacher.teacher_id = teaches.teaches_id
    AND student.student_id = takes.takes_id
    AND takes.course_id = teaches.course_id
    AND takes.sec_id = teaches.sec_id
```

```

AND takes.semester = teaches.semester
AND takes.takes_year = teaches.teaches_year
GROUP BY teacher.teacher_name, student.student_name
HAVING COUNT(*) >= 2;

```

4. Pour chaque enseignant, afficher tous les étudiants qui ont suivi plus de deux cours dispensés par cet enseignant, ainsi que le nombre total de cours suivis par chaque étudiant, sans utiliser la clause HAVING.

```

SELECT T.teachername, T.studentname, T.totalcount
FROM (
  SELECT
    teacher.teacher_name AS teachername,
    student.student_name AS studentname,
    COUNT(*) AS totalcount
  FROM teacher
  JOIN teaches ON teacher.teacher_id = teaches.teaches_id
  JOIN takes ON takes.course_id = teaches.course_id
    AND takes.sec_id = teaches.sec_id
    AND takes.semester = teaches.semester
    AND takes.takes_year = teaches.teaches_year
  JOIN student ON student.student_id = takes.takes_id
  GROUP BY teacher.teacher_name, student.student_name
) T
WHERE T.totalcount >= 2
ORDER BY T.teachername;

```

5. Afficher les identifiants et les noms des étudiants qui n'ont pas suivi de cours avant 2010.

```

SELECT student.student_id, student.student_name
FROM student
MINUS

```

```
SELECT student.student_id, student.student_name
FROM student
JOIN takes ON takes.takes_id = student.student_id
WHERE takes.takes_year < 2009;
```

ID	NAME
76543	Brown
12345	Shankar
19991	Brandt
98988	Tanaka
45678	Levy
76653	Aoi
23121	Chavez
55739	Sanchez
00128	Zhang
00015	DeVries

6. Afficher tous les enseignants dont les noms commencent par "E".

```
SELECT * FROM teacher WHERE teacher_name LIKE 'E%';
```

ID	NAME	DEPT_NAME	SALARY
22222	Einstein	Physics	95000
32343	El Said	History	60000

2 rows returned in 0.01 seconds [Download](#)

7. Afficher les salaires et les noms des enseignants qui perçoivent le quatrième salaire le plus élevé.

```
SELECT name, salary
FROM teacher t1
WHERE 3 = (
    SELECT COUNT(DISTINCT t2.salary)
    FROM teacher t2
    WHERE t2.salary > t1.salary
);
```

NAME	SALARY
Gold	87000

1 rows returned in 0.01 seconds [Download](#)

8. Afficher les noms et les salaires des trois enseignants qui perçoivent les salaires les moins élevés. Les afficher par ordre décroissant.

```
SELECT name, salary
FROM teacher t1
WHERE 2 >= (
    SELECT COUNT(DISTINCT t2.salary)
    FROM teacher t2
    WHERE t2.salary < t1.salary
)
ORDER BY salary DESC;
```

Results

Explain

Describe

Saved SQL

History

NAME	SALARY
Califleri	62000
El Said	60000
Mozart	40000

3 rows returned in 0.01 seconds

Download

9. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause IN.

```
SELECT s.name
FROM student s
WHERE s.ID IN (
    SELECT tk.ID
    FROM takes tk
    WHERE tk.semester = 'Fall' AND tk.year = 2009
);
```

Results	Explain	Describe	Saved SQL	History
NAME				
Zhang				
Shankar				
Peltier				
Levy				
Williams				
Brown				
Bourikas				
7 rows returned in 0.03 seconds				

10. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause SOME.

```
SELECT s.name
FROM student s
WHERE 'Fall' = SOME (
    SELECT tk.semester
    FROM takes tk
    WHERE tk.ID = s.ID AND tk.year = 2009
);
```

Results	Explain	Describe	Saved SQL	History
NAME				
Zhang				
Shankar				
Peltier				
Levy				
Williams				
Brown				
Bourikas				
7 rows returned in 0.03 seconds Download				

11. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la jointure naturelle (NATURAL INNER JOIN).

```
SELECT DISTINCT s.name
FROM takes tk
NATURAL JOIN student s
WHERE tk.semester = 'Fall' AND tk.year = 2009;
```

Results	Explain	Describe	Saved SQL	History
NAME				
Brown				
Zhang				
Levy				
Bourikas				
Shankar				
Peltier				
Williams				
7 rows returned in 0.04 seconds Download				

12. Afficher les noms des étudiants qui ont suivi un cours en automne 2009, en utilisant la clause EXISTS.

```

SELECT s.name
FROM student s
WHERE EXISTS (
    SELECT 1
    FROM takes tk
    WHERE tk.ID = s.ID AND tk.semester = 'Fall' AND tk.year = 2009
);

```

Results	Explain	Describe	Saved SQL	History
NAME				
Zhang				
Shankar				
Peltier				
Levy				
Williams				
Brown				
Bourikas				
7 rows returned in 0.03 seconds Download				

13. Afficher toutes les paires d'étudiants qui ont suivi au moins un cours ensemble.

```

SELECT s1.name AS student1, s2.name AS student2
FROM takes tk1
JOIN takes tk2 ON tk1.course_id = tk2.course_id
    AND tk1.sec_id = tk2.sec_id
    AND tk1.semester = tk2.semester
    AND tk1.year = tk2.year
    AND tk1.ID < tk2.ID
JOIN student s1 ON tk1.ID = s1.ID
JOIN student s2 ON tk2.ID = s2.ID
ORDER BY s1.name, s2.name;

```

STUDENT1	STUDENT2
Brown	Bourikas
Levy	Bourikas
Levy	Brown
Levy	Williams
Shankar	Bourikas
Shankar	Bourikas
Shankar	Brown
Shankar	Levy
Shankar	Williams
Shankar	Williams

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.04 seconds [Download](#)

elgaranihassna@gmail.com bdd2003 en Copyright © 1999, 2024, Oracle and/or its affiliates. Oracle APEX 24.2.2

Exercice n°2 :

1) Avec les dépendances $A \rightarrow B$ et $B \rightarrow C$, A est la clé primaire. La relation est en 1NF et 2NF, mais pas en 3NF ni BCNF à cause de la dépendance transitive $B \rightarrow C$. Il faut décomposer en deux relations pour atteindre la 3NF.

Pour corriger cela et atteindre la 3NF, il est nécessaire de scinder la relation en deux : R1(A, B) et R2(B, C), chacune étant alors conforme à la 3NF.

2) Avec $A \rightarrow B$ et $A \rightarrow C$, A est la clé primaire. La relation respecte les formes normales 1NF, 2NF, 3NF et BCNF, elle est donc déjà bien normalisée.

3) Avec $A, B \rightarrow C$ et $C \rightarrow B$, il y a une dépendance partielle qui viole la 2NF. La relation n'est pas en 2NF, 3NF ni BCNF. La décomposition en deux relations permet d'atteindre la 2NF.

Pour remédier à ce problème, il faut décomposer la relation en deux sous-relations : R1(A, B, C) pour la dépendance $A, B \rightarrow C$, et R2(C, B) pour la dépendance $C \rightarrow B$. Ces deux relations sont alors conformes à la 2NF.